The Course TA-Bot: Retrieval and Memory

ECE 6256 / CS 6604 · Assignment 3 · Revised November 6, 2026

1. System and real-model experiment

The bot retrieves from 108 papers and 5,711 supplied chunks using SQLite FTS5 BM25. Title terms receive weight 5. From 40 candidate chunks, paper ranks sum the best two scores; up to three passages are retained for each of the top three papers. Original and rewritten searches are combined by reciprocal rank fusion (damping 60). Generation receives attributed evidence, up to three previous exchanges, and a filtered student profile. The prompt requests 100–250 words, paper titles and IDs, and explicit abstention when evidence is incomplete; the answer budget is 800 tokens. Returned citation IDs come from supplied retrieval evidence, not from parsing model text.

The archived experiment uses **meta-llama/Llama-3.1-8B-Instruct**, temperature 0 and seed 42. The prior team draft describes serving it with vLLM on a PACE-ICE A40, rather than the staff's shared endpoint. Model/seed/results are saved; hardware details and earlier public-dev logs survive only as descriptions in that draft. This revision audits those saved results and reruns local checks; it does not claim a new real-model or course-judge run.

2. Ablation: the rewrite flag (R4)

Design. Forty authored two-turn dialogues are split into 20 tuning and 20 labelled held-out cases. The reported split contains 15 context-dependent follow-ups and five standalone topic switches. Only **rewrite** changes; corpus, retrieval parameters, prompts, model and seed are fixed. Each condition starts a fresh bot. A hit means the final turn's citations contain the specified gold paper. The 20 cases yield 40 condition-dialogue runs, with no recorded endpoint failures. The flag activates both query rewriting and last-subject fallback; it is not a pure rewriting intervention.

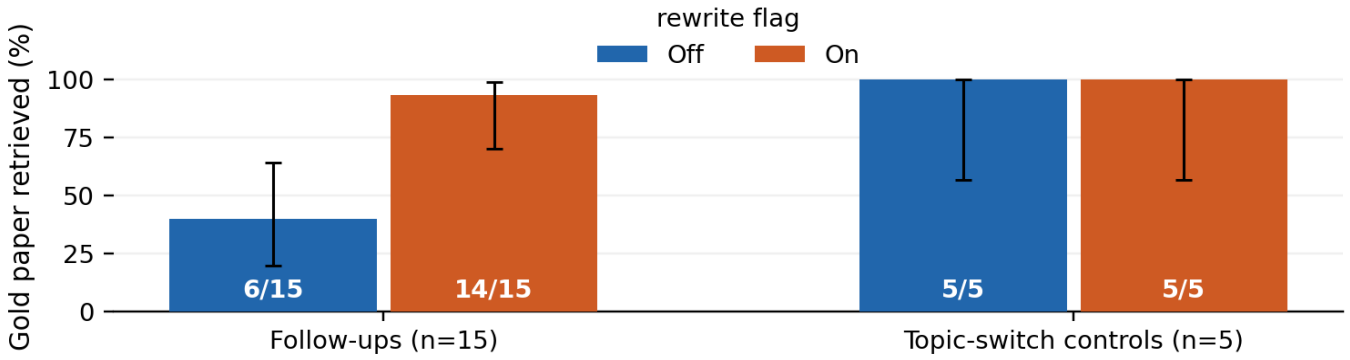

Figure 1. Archived real-model outcomes; error bars are 95% Wilson intervals recomputed from the saved counts. Unlike the original bootstrap plot, these retain uncertainty for five successes out of five. On/off observations are paired by dialogue; significance is assessed with exact McNemar, not overlap of marginal intervals.

Group	Flag off	Flag on	Paired gains / losses	Exact p
Follow-ups	6/15 (40.0%)	14/15 (93.3%)	8 / 0	0.0078125
Topic-switch controls	5/5 (100%)	5/5 (100%)	0 / 0	1.0

Result. Follow-up recall improves by **53.3 percentage points** (paired percentile-bootstrap 95% interval 26.7–80.0 points; 2,000 resamples, seed 42). The controls show no observed regression, but 5/5 has a Wilson interval of 56.6–100%; it cannot establish safety on arbitrary topic switches.

3. What produced the gain?

Two sufficient routes on these examples. In the real-model records, all eight newly recovered gold papers occur at rank 2 or 3. The code appends last-subject fallback only after the fused top three, so these recoveries entered through fused retrieval, rather than through the append step. A new replay of the same split with the canned mock also gives 6/15 versus 14/15, with the same eight recoveries all at rank 4. The flag-off citation lists match the archived real run exactly. Thus fallback alone is sufficient for the observed recall gain, while real rewriting recovers those papers within the top three. This diagnostic does not establish an additional recall benefit of rewriting over fallback, or any answer-quality advantage from a higher rank.

Failure and validity. The remaining miss, followup-18, asks 'What is BERT?' then 'Which datasets showed the largest gains?' The second question has no recognized back-reference, so the contextual branch is skipped. The constructed follow-ups all concern the first turn's gold paper, favoring subject carry-over. The prior team draft also reports replaying the labelled held-out split during development: it is not a pristine test set. The sample is small, with one model and one seed. These results support contextual retrieval on this development set, not a general claim that query rewriting alone adds 53.3 points or meets the course's answer-quality threshold.

4. Memory design and privacy (R3)

Store and update. SQLite facts are keyed by user_id and contain one of five allowlisted categories: name, project, interest, academic background, and response preference. Explicit user statements trigger deterministic extraction before retrieval; assistant output and paper text never create facts. The latest name/project replaces the previous value. Other categories can coexist, duplicate facts are ignored, and oldest rows are evicted above 30 facts per user; individual facts are limited to 200 characters. Direct recall reads the store without generation. Project-related search can incorporate accepted profile facts when the question refers to the user's work.

Refused content	Reason for refusal
Secrets, account/contact identifiers, emails, long digit runs	No tutoring benefit; high exposure cost
Health/protected attributes, finances, grades, misconduct	Sensitive information not needed for teaching
Third-party/quoted claims, hypotheticals, negation, temporary states	Not reliable, durable facts about this user
Opt-outs and recognizable instruction injection	Respect retention choices; limit stored-prompt injection

Isolation and control. An opt-out applies to the entire message and overrides an explicit request to remember. A sensitive sentence is vetoed even if it also contains a safe preference. History is scoped by (user_id, session_id), limited to three exchanges, and refused questions/responses are redacted before replay. Category forgetting and full reset also clear that user's histories, preventing deleted facts from returning through conversational context. A fresh session retains only that user's accepted profile. The optional memory_path persists facts across restarts; otherwise the store is in process memory.

Limits. English regexes can miss sensitive paraphrases or reject harmless research topics; passing negative probes is not a privacy proof. Preferences can become contradictory because they are not replacement categories. SQLite is unencrypted and user_id is trusted rather than authenticated. Refusal prevents retained profile facts and replayed turns; the current message can still reach the configured endpoint.

5. Validation, provenance and reproducibility

Evidence	Result and status
Archived real-model ablation	15 follow-ups + 5 controls; 0/40 failed condition runs
Real public-dev results in prior draft	Reported: 10/10 single-turn; 3/3 dialogue endings; 2/2 public and 10/10 extra memory probes. Raw logs not committed.
New local replay for this revision	49/49 offline tests; mock dev: 100% single-turn and dialogue-ending recall, 2/2 public memory probes. The suite also checks all 10 extra probes.
Repeatability in prior real-model draft	Reported: equal dev scores across two runs, one citation-order change; raw repeat logs unavailable.
Course judge / answer-quality threshold	Not measured in the archived evidence; no bonus judge-error claim.

The current 49-test suite covers interface signatures, paper-level citations, read-only corpus access, alternate working directories, cross-thread calls, endpoint failures, session/user isolation, persistent memory, sensitive-fact refusal, deletion, and deterministic citation membership. It passes in a fresh Python 3.10.12 environment. Disabling SQLite's thread check permits calls from another thread; the tests do not establish safe concurrent writes. The team relaxed a real-model order-sensitive test after seeing equal citation sets in different orders; this revision reruns that check only offline, so it does not claim the revised test passed on the real model.

Reproduce. Install requirements.txt, run the commands below, and supply the course/development endpoint and its served model explicitly for a new real run. Use --out to preserve the archived real-model results. The report adds a mock diagnostic, fresh check logs, recomputed statistics, a vector plot, and SHA-256 source/data hashes under report/. The real-model artifact lacks raw answers, rewrite text, server settings and run-time code/corpus hashes; those were not retroactively reconstructed as execution provenance.

```
python -m unittest discover -s tests -v
python eval_dev.py --mock
python eval_dev.py --endpoint "$A3_ENDPOINT" --model "$A3_MODEL"
python ablation/run_ablation.py --endpoint "$A3_ENDPOINT" \
  --model "$A3_MODEL" --split heldout --out /tmp/real-ablation.json
python report/build_report.py
```

6. Answer quality and judging

Retrieval hit-rate measures whether the supplied evidence contains a gold paper; returning that ID does not verify the answer's claims or use of the evidence. The prior team draft describes answer inspections, a stand-in Llama scorer, and individual scoring errors, but the corresponding answers/scores are not archived. The local scorer is not the course judge. No numeric quality score, 0.76-threshold pass, or documented course-judge mistake is claimed. Real-model answer grounding, privacy leakage and graded-server performance still need direct evaluation.

7. AI-use disclosure

The prior team draft documents Claude Code (Anthropic) assistance with core retrieval/memory code, prompts, tests, PACE experiments, analysis and the initial report. OpenAI Codex assisted with compatibility/privacy fixes, local validation and tooling in the earlier revision. For this revision, Codex pulled and inspected the collaborator's changes, checked archived per-dialogue outcomes against their labels, recomputed statistics, replayed the mock diagnostic, ran the offline suite, and wrote/rendered this report. The real-model run was supplied by the collaborator and was not rerun here. Verification described here is automated and linked to saved artifacts; no undocumented independent human review or course-judge evaluation is represented as completed.